Name - Saikat Sheet
Batch - 25SUB5936_Linux System Programming
ID - 68500

Assignment 1: Design Pattern Explanation - Prepare a one-page summary explaining the MVC (Model-View-Controller) design pattern and its two variants. Use diagrams to illustrate their structures and briefly discuss when each variant might be more appropriate to use than the others.

> The **Model-View-Controller (MVC)** pattern is a foundational architectural design used to separate an application's concerns into three distinct components. This separation ensures that the business logic, user interface, and user input handling can evolve independently, which is crucial for maintaining large-scale software systems.

The Core Components

- **Model:** Manages the application's data and business logic. It responds to requests for information and updates its state based on instructions from the Controller.
- **View:** The visual representation of the data (the UI). It "observes" the Model and renders the information for the user.
- **Controller:** Acts as the intermediary. It processes user inputs (clicks, keystrokes), converts them into commands for the Model, and may occasionally instruct the View to change.

Variant 1: Traditional (Active) MVC

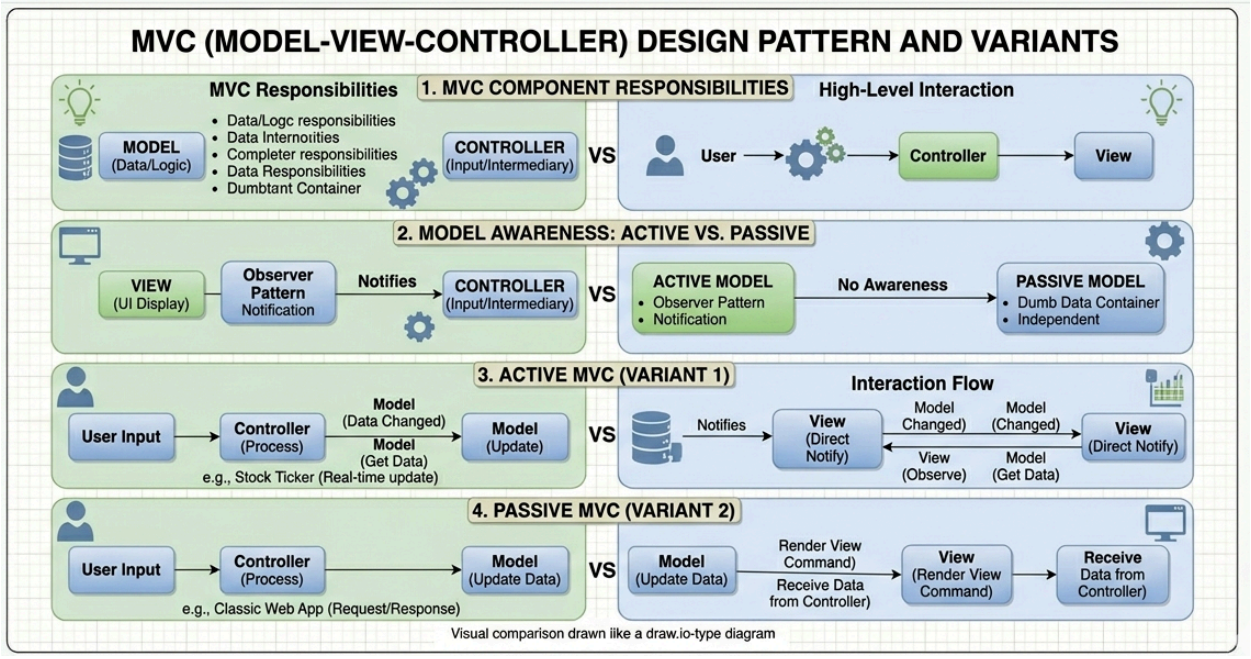
In the traditional or "Active" MVC pattern, the Model is responsible for notifying the View whenever its data changes. This is often implemented using the **Observer Pattern**.

- **Structure:** The View subscribes to the Model. When the Model's data updates, it broadcasts a notification, and the View automatically refreshes itself to display the new state.
- **Best Use Case:** Best for **real-time applications** (e.g., a stock ticker or a collaborative document) where the UI must reflect data changes instantly without the user or Controller constantly asking for updates.

Variant 2: Passive MVC

In the "Passive" MVC pattern, the Model has no awareness of the View or the Controller. It is a "dumb" data container that only changes when told to do so.

- **Structure:** The Controller is the "brain." It updates the Model, then explicitly tells the View to refresh. The View never talks to or observes the Model directly; it receives all its data and instructions from the Controller.
- **Best Use Case:** Ideal for **Web Applications** (like standard HTML-based sites). Since the server cannot "push" a refresh to a browser easily, the Controller handles the logic of saving data and then rendering a new page or partial view for the user.



Assignment 2: Principles in Practice - Draft a one-page scenario where you apply Microservices Architecture and Event-Driven Architecture to a hypothetical e-commerce platform. Outline how SOLID principles could enhance the design. Use bullet points to indicate how DRY and KISS principles can be observed in this context.

> **The "SwiftShop" E-Commerce Modernization**

The Context:

SwiftShop is a rapidly growing e-commerce platform transitioning from a slow, "all-in-one" legacy system to a modern architecture. To handle high-traffic sales events (like Black Friday), SwiftShop is moving to a **Microservices** and **Event-Driven Architecture (EDA)**.

1. Applying Microservices & Event-Driven Architecture

In this new design, SwiftShop is broken down into independent services:

- **Inventory Service:** Manages stock levels.
- **Order Service:** Handles customer purchases.
- **Notification Service:** Sends emails and SMS.
- **Payment Service:** Processes transactions.

The Event-Driven Flow:

Instead of the Order Service waiting for the Payment Service to finish (which could cause timeouts), we use an **Event Bus** (like RabbitMQ or Kafka):

1. A customer clicks "Buy." The **Order Service** creates an order and publishes an event: `Order_Created`.
2. The **Payment Service** and **Inventory Service** are "listening." They see the event and immediately start processing payment and reserving stock simultaneously.
3. Once payment is successful, a `Payment_Processed` event is published, which triggers the **Notification Service** to send a confirmation email.

2. Enhancing the Design with SOLID Principles

Applying SOLID ensures that as SwiftShop adds new features (like "Buy Now, Pay Later"), the code doesn't break.

- **Single Responsibility (S):** Each microservice does *one* thing. The Notification Service only handles messaging; it doesn't care about how the payment was processed.
- **Open/Closed (O):** We can add a new "Loyalty Points Service" to listen for `Payment_Processed` events without changing any existing code in the Payment or Order services.
- **Liskov Substitution (L):** If we have multiple payment gateways (Stripe, PayPal), they all implement a common `IPaymentProvider` interface, allowing them to be swapped without affecting the Payment Service logic.
- **Interface Segregation (I):** The Inventory Service shouldn't have to implement a giant "Product" interface that includes marketing descriptions and reviews; it only needs an `IStockItem` interface with `Quantity` and `SKU`.
- **Dependency Inversion (D):** High-level business logic (Order Processing) depends on abstractions (Interfaces), not on specific database drivers or third-party APIs.

---

3. Observing DRY and KISS Principles

DRY (Don’t Repeat Yourself)

- **Shared Libraries:** Use a shared "Event Schema" library so that every microservice defines an `Order_Created` event in the exact same way, preventing inconsistent data formats.
- **Centralized Authentication:** Instead of every service writing its own login logic, a single **API Gateway** handles authentication for the entire platform.

KISS (Keep It Simple, Stupid)

- **Avoiding Over-Engineering:** Avoiding building a complex "Global AI Search" if a simple database query for "Product Name" meets the current user's needs.

- **Small Service Scopes:** Keeping microservices small. If a service becomes too hard to explain in one sentence, it’s too complex and should be split.
- **Standard Tech Stack:** Use well-known, standard libraries for the Event Bus rather than building a custom, "clever" messaging protocol from scratch.

Assignment 3: Trends and Cloud Services Overview - Write a three-paragraph report covering: 1) the benefits of serverless architecture, 2) the concept of Progressive Web Apps (PWAs), and 3) the role of AI and Machine Learning in software architecture. Then, in one paragraph, describe the cloud computing service models (SaaS, PaaS, IaaS) and their use cases.

>

1. Benefits of Serverless Architecture

- **Operational Efficiency:** Eliminates the need for developers to provision, manage, or maintain physical or virtual servers.
- **Automatic Scaling:** The cloud provider automatically scales resources up or down based on real-time demand.
- **Cost Optimization:** Uses a "pay-as-you-go" model where you only pay for the exact duration your code runs, preventing costs from idle servers.
- **Focus on Innovation:** Frees development teams to focus entirely on writing business logic and features rather than backend infrastructure.

2. Progressive Web Apps (PWAs)

- **Hybrid Experience:** Combines the best features of standard websites and native mobile applications.
- **Native Functionality:** Supports native-like features such as offline access, push notifications, and home screen installation.
- **Platform Independence:** Built with HTML, CSS, and JavaScript, allowing a single codebase to work across all devices and browsers.

- **Improved Accessibility:** Eliminates the need for app store downloads, reducing friction for new users and lowering development costs.

3. AI and Machine Learning in Software Architecture

- **AI-as-a-Service:** Enables the integration of pre-built ML models (like natural language processing or predictive analytics) via simple APIs.
- **Data-Centric Design:** Requires architectures capable of handling massive data pipelines and real-time processing to feed ML models.
- **Adaptive Systems:** Software becomes more intuitive and personalized, capable of optimizing its own performance based on user behavior.
- **Automated Decision-Making:** Embeds intelligence directly into the application flow to automate complex tasks and provide smarter user experiences.

4. Cloud Computing Service Models

- **IaaS (Infrastructure as a Service):** Provides raw computing blocks like virtual servers and storage.
    - **Use Case:** Ideal for hosting websites or performing high-performance big data analysis where total infrastructure control is needed.
- **PaaS (Platform as a Service):** Provides a framework and tools for developing and testing applications.
    - **Use Case:** Best for streamlining development workflows by removing the need to manage operating systems or middleware.
- **SaaS (Software as a Service):** Ready-to-use software delivered over the internet on a subscription basis.
    - **Use Case:** The standard choice for business productivity tools like Gmail, Slack, or Salesforce.

Exercise 1: Containerizing with Docker
Objective: Package an application program into a Docker container.(May use the client-server application written in capstone)
Tasks:

- Write a Dockerfile that specifies the base image and includes the necessary dependencies and command to run your microservice.
- Build the Docker image using the Docker CLI.
- Run a container from your image and test the microservice via the exposed port.

Commands

Creating a calculator

> Creating Dockerfile inside the project folder

—----------

```
FROM ubuntu:22.04
RUN apt update && apt install -y g++
COPY server.cpp / server.cpp
RUN g++  /server.cpp  -o  /server
EXPOSE  9090
CMD [“/server”]
```

—----------

> Creating a server.cpp file

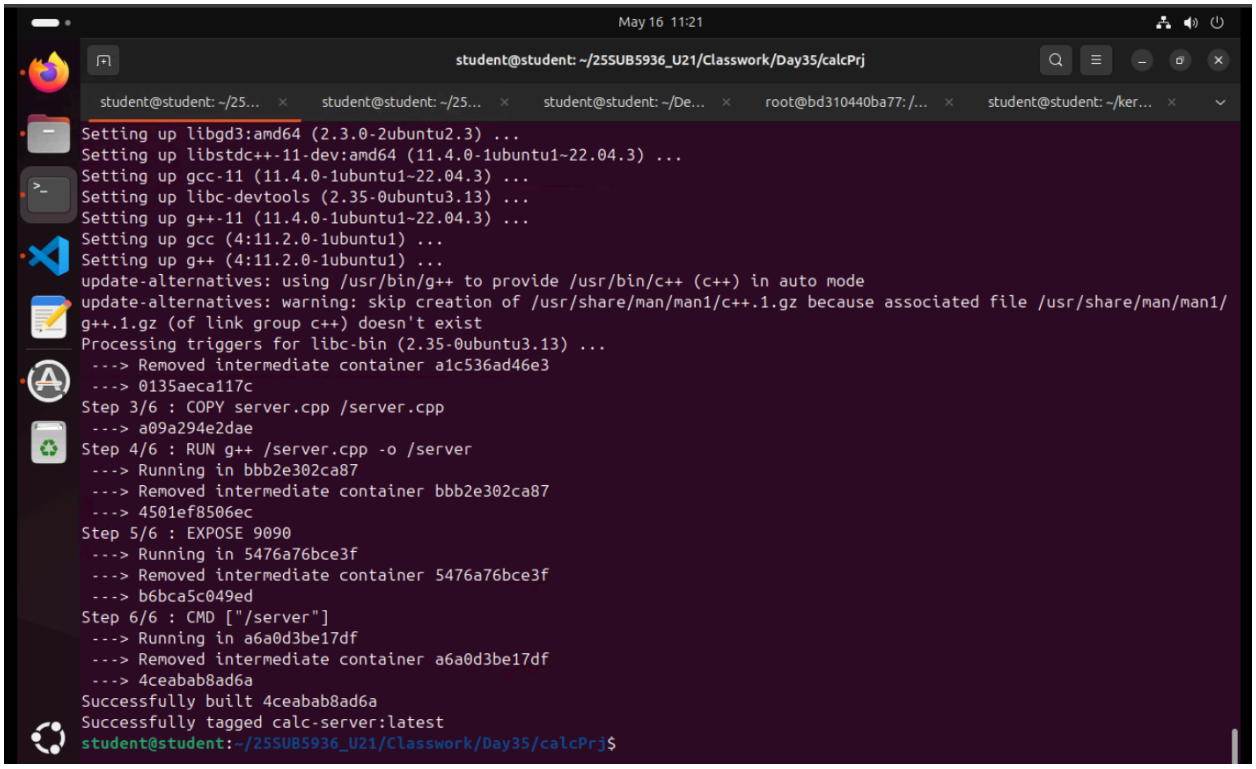
—----------

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
   int server_fd, client_fd;
   struct sockaddr_in addr;
```

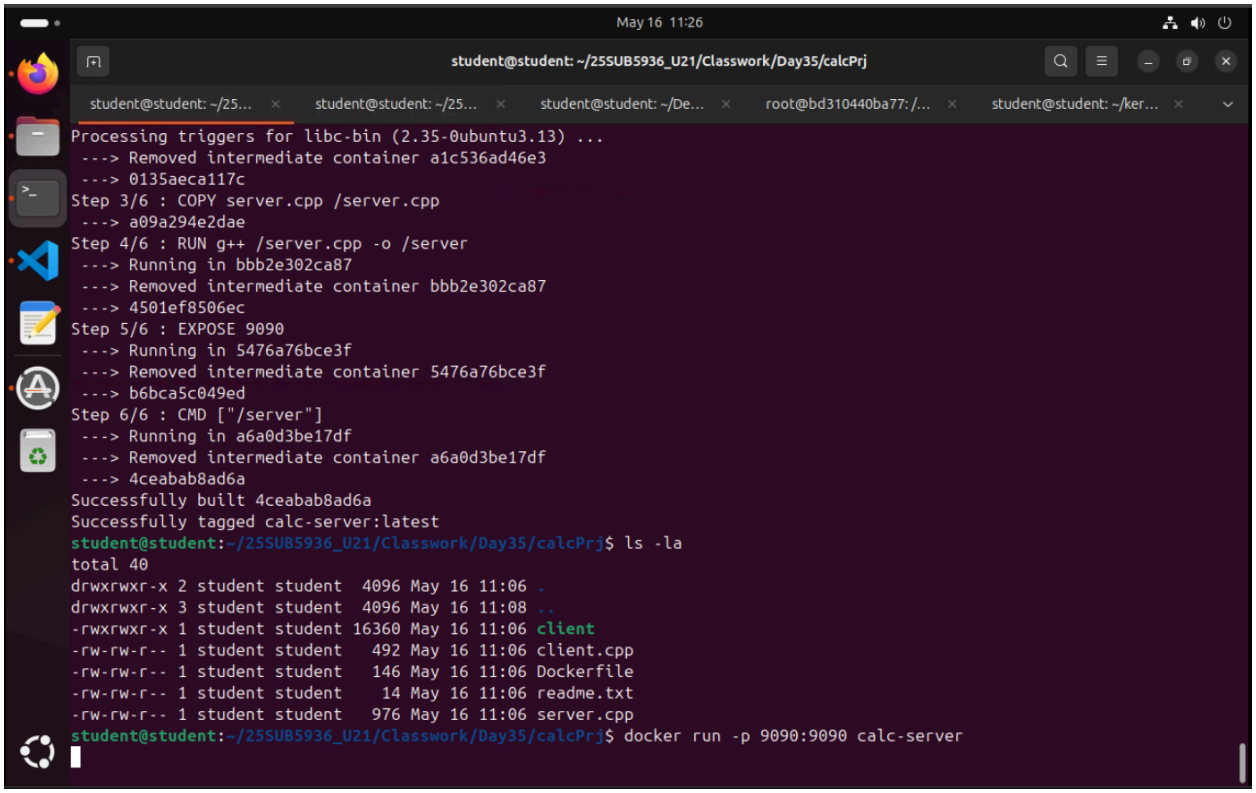
```
    char buffer[1024];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(9090);
    addr.sin_addr.s_addr = INADDR_ANY;

    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 1);   // ⚠️ only 1 connection

    std::cout << "Calculator server running...\n";

    while (1) {
        client_fd = accept(server_fd, NULL, NULL);
        memset(buffer, 0, sizeof(buffer));

        read(client_fd, buffer, sizeof(buffer));

        int a, b;
        sscanf(buffer, "%d %d", &a, &b);

        int result = a + b;
        sprintf(buffer, "Result: %d\n", result);

        write(client_fd, buffer, strlen(buffer));
        close(client_fd);

        sleep(2); // ⚠️ simulate slow processing
    }
}
```

—----------

[fetching the docker file and creating calc server inside the project folder]
> docker build -t calc-server .



[To invoke docker we have to run a command]
> docker run -p 9090:9090 clac-server



[To connect with the docker and application we have to write client code]

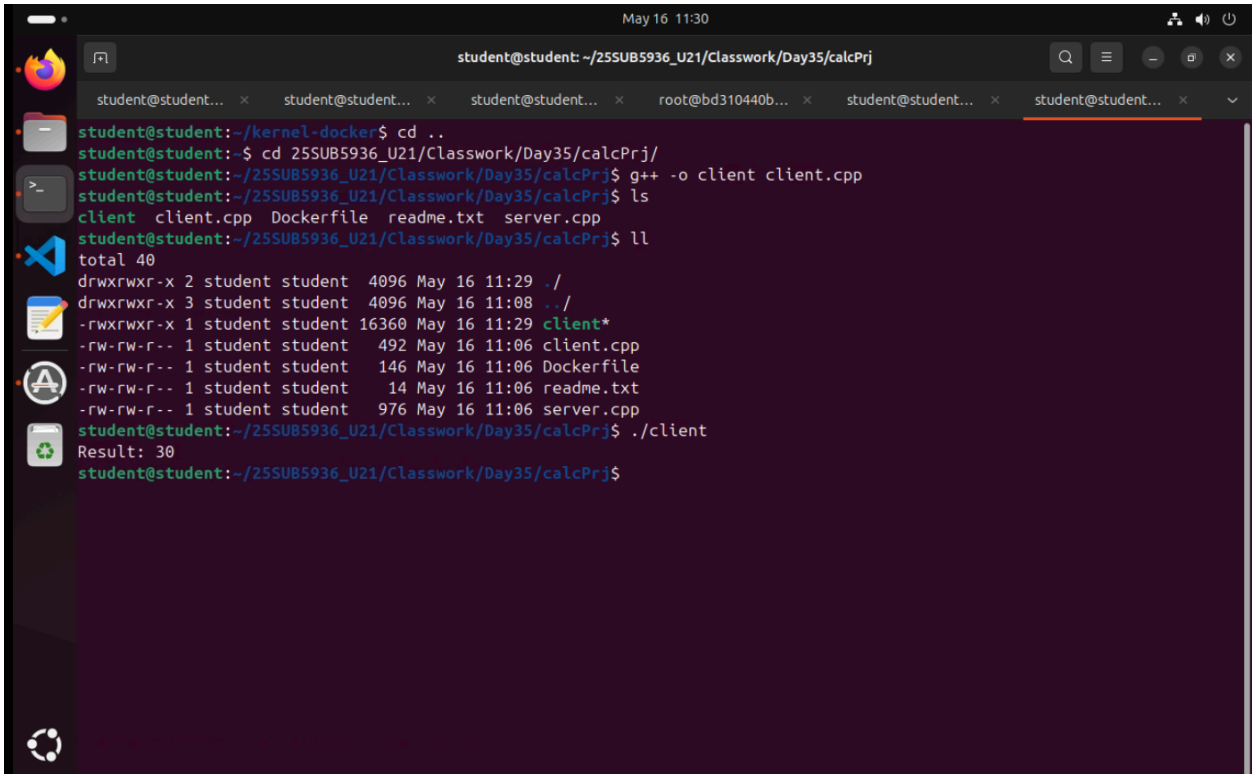
—----------

```
#include <arpa/inet.h>
#include <unistd.h>
#include <cstring>

int main() {
   int sock = socket(AF_INET, SOCK_STREAM, 0);
   sockaddr_in addr{};
   addr.sin_family = AF_INET;
   addr.sin_port = htons(9090);
   addr.sin_addr.s_addr = inet_addr("127.0.0.1");

   connect(sock, (sockaddr*)&addr, sizeof(addr));
   write(sock, "10 20", 5);

   char buf[100];
        memset(buf,'\0',100);
   read(sock, buf, sizeof(buf));
   write(1, buf, strlen(buf));
   close(sock);
}
```

—----------

[Now we have to compile the client application]

```
> g++  -o client client.cpp
> ./client
```

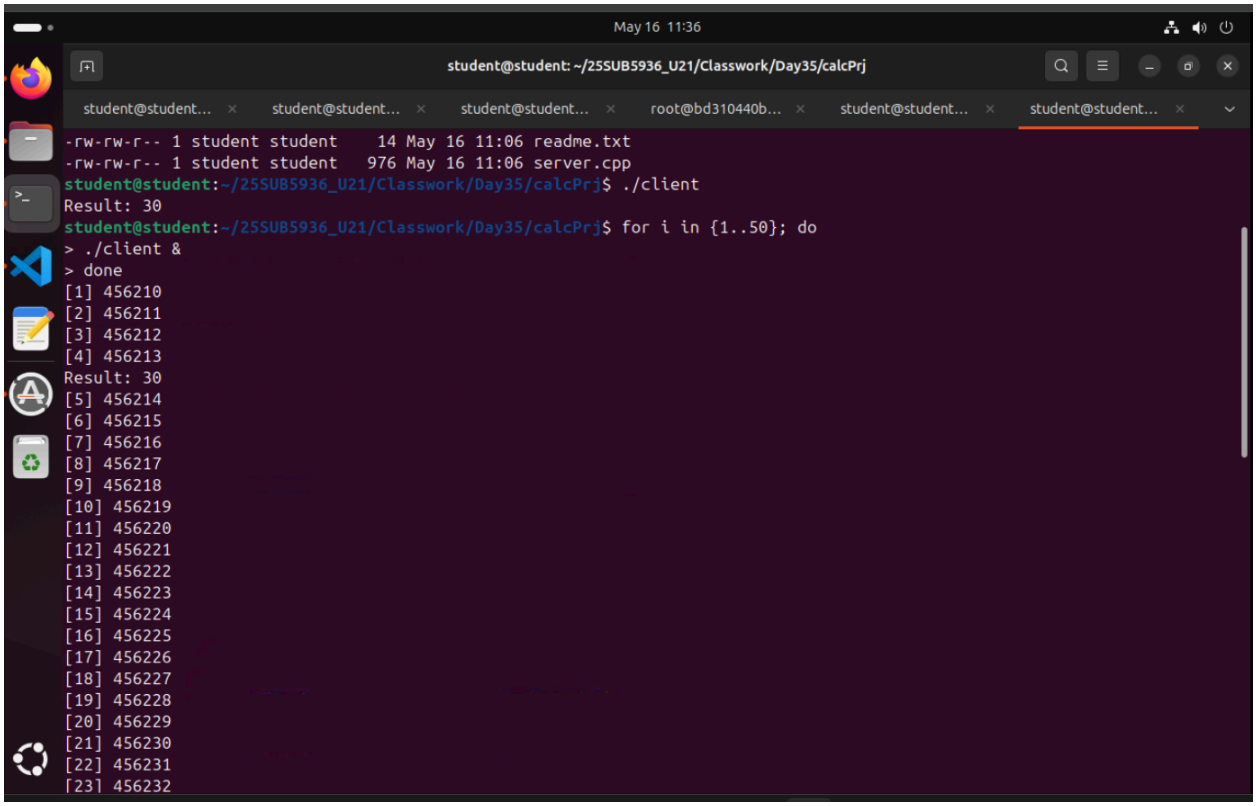


[To run multiple clients we have to use a looping statement]

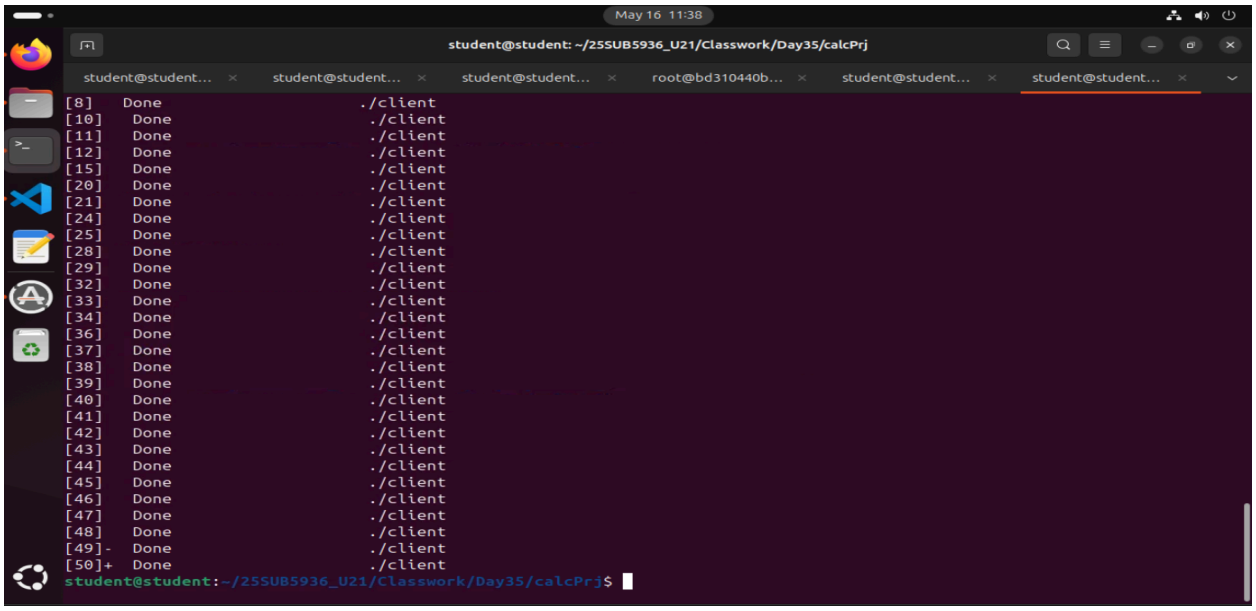
> for i in {1..50}; do

> ./client &

> done



[Now we can see how many clients are served]

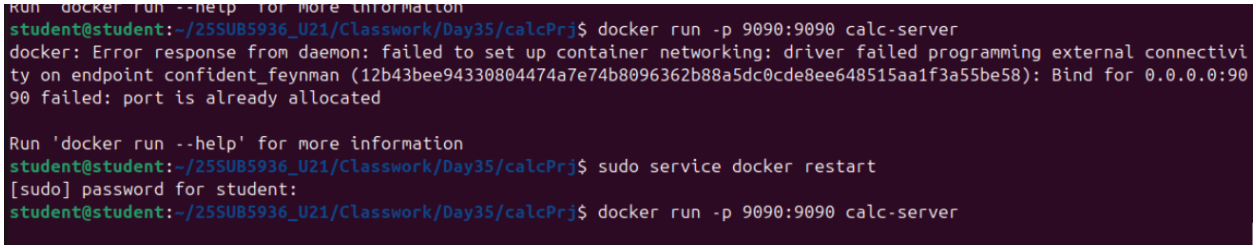


Exercise 2: Orchestration with kubernetes
Objective: Deploy your containerized microservice on a local kubernetes cluster using Minikube and expose it.
Tasks:

- Install Minikube and start a local Kubernetes cluster.
- Create a Kubernetes Deployment YAML file to define your application deployment.
- Use kubectl to apply the deployment and manage the cluster state.
- Expose the microservice using a kubernetes Service and test it using the Minikube service URL.

Commands:

[Starting the server]
> docker run -p 9090:9090 calcc-server



[Starting Minikube]
> minikube start

[Pointing the terminal to Minikube's Docker daemon]
> eval $(minikube docker-env)

[Image building]
> docker build -t calc-server .

[Using `kubectl` to send my configurations to the cluster]
> kubectl apply -f deployment.yaml
> kubectl apply -f service.yaml

—----------

deployment.yaml

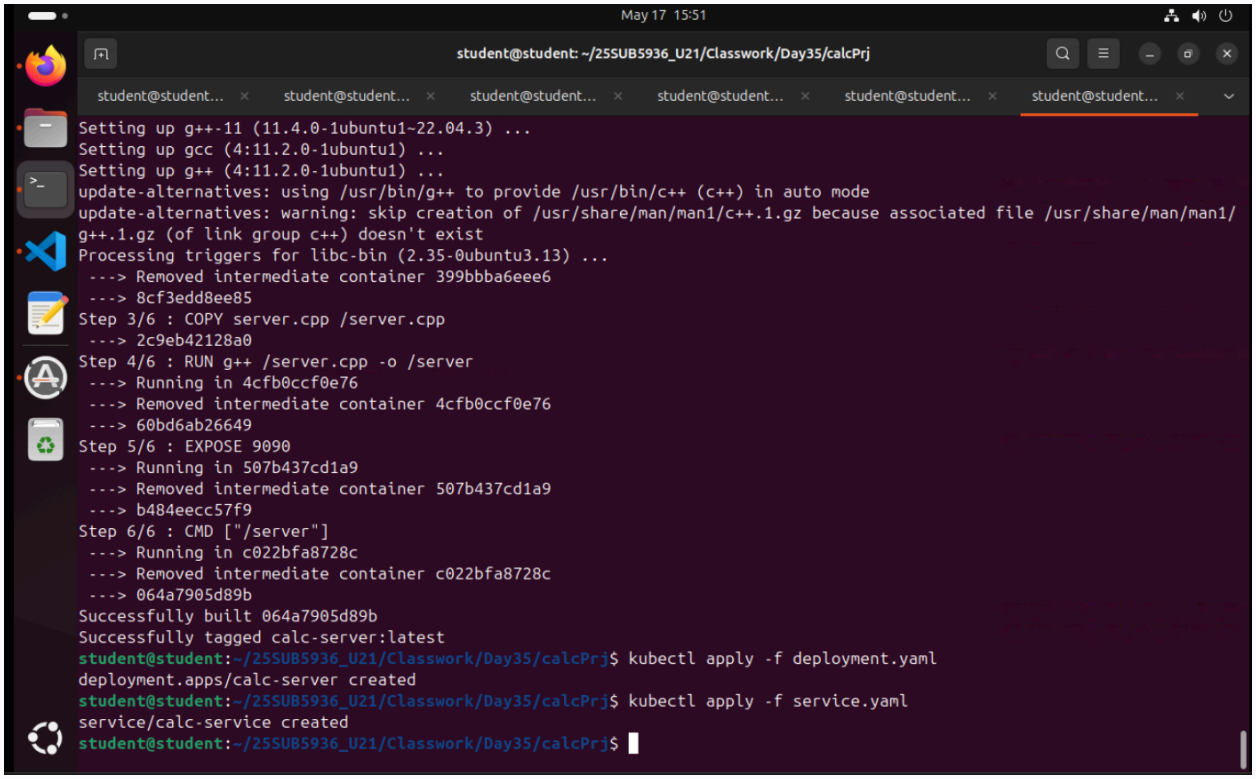
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: calc-server
spec:
  replicas: 3   # THIS is the fix
  selector:
    matchLabels:
      app: calc
  template:
    metadata:
      labels:
        app: calc
    spec:
      containers:
      - name: calc
        image: calc-server
        ports:
        - containerPort: 9090
```

—----------

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: calc-service
spec:
  selector:
    app: calc
  ports:
    - port: 9090
```

targetPort: 9090



[Checking if my 3 replicas (pods) are running]
> kubectl get pods
